

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 06/Unit 06: Constructors, Overloading and Destructors

Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Review of Last Week's Content: Classes and Objects
2. A deeper look at constructors
3. Constructors: Initializer Lists
4. Overloading Constructors
5. Destructors
6. Summary.



Classes and Objects

- Recall from your last session on classes and objects.
- A class in Object Oriented Programming is a collection of
 - **data**, and
 - the **operations** on that data.
- In terms of C++ programming, a class consists of variables that may be primitive data types or classes, as well as functions (on these variables).
- In order to initialize a class, you need to invoke its constructors.



Constructors

- A constructor is a member function that is executed automatically whenever an object is created.
- The term constructor is sometimes abbreviated `ctor`, especially in comments in program listings.



Constructors

- Note that the name of the constructor is that of the class itself.
- Constructors do not have a return type.
- There are several ways to initialize variables using a constructor.
- One way is to use a ":" after the constructor's name, followed by the name of each variable and then its initial value in parenthesis.

```
class Counter
{
    private:
        unsigned int count;
    public:
        Counter() : count(0)
```



Constructors: Initializer Lists

- If multiple members must be initialized, they're separated by commas. The result is the **initializer list** (sometimes called by other names, such as the member-initialization list).

```
someClass() : m1(7), m2(33), m2(4) ←———— initializer list
{ }
```

- Another way to initialize values in a constructor is by assigning the values in the body of the constructor.



Constructors: Initialization

- Another way to initialize variables in the constructor is by assignment.
- Have a look at the following example:

```
class circle {  
    private:  
        double radius, area, circum;  
    public:  
        circle(double r)  
        {    radius = r; }  
  
        void calc_circumference() {  
            circum = 2 * 3.1416 * radius;  
        }  
  
        void calc_area() {  
            area = 3.1416 * radius * radius;  
        }  
}
```



Constructors: Initialization

```
void print_circle() {  
    calc_circumference();  
    calc_area();  
    cout << "Circle: Radius = " << radius <<  
    ", Circumference = " << circum << ", Area = "  
    << area << endl;  
}  
};  
  
int main(void) {  
    circle c1(4.0), c2(5.0), c3(1.0);  
  
    c1.print_circle();  
    c2.print_circle();  
    c3.print_circle();  
}
```




Constructors and_INITIALIZER Lists

- Consider the following class Counter →
- It has a constructor with no parameters.
- The associated initializer list initializes count to 0.
- This constructor is called automatically whenever a new object of type Counter is created.
- Thus in main() the statement
 Counter c1, c2;
creates two objects of type Counter.

```
class Counter
{
    private:
        unsigned int count;
    public:
        Counter() : count(0)
            { /*empty body*/ }
        void inc_count()
            { count++; }
        int get_count()
            { return count; }
};
```



Constructors: Recap

- Constructors have the same name as the class.
- A constructor has no return type associated with it.
- Earlier we saw that a constructor can be initialized using either an initializer list, or by assigning values to instance variables in the body of the constructor.
- To initialize multiple values, use the following approach:

```
someClass() : m1(7), m2(33), m2(4) ←———— initializer list
{ }
```



Initializer Lists

- The initializer list is the preferred way of initializing an object using a constructor.
- There are several reasons such as, members initialized in the initializer list are given a value before the constructor even starts to execute.
- This is important in some situations such as the initializer list is the only way to initialize const member data.
- Actions more complicated than simple initialization must be carried out in the constructor body, as with ordinary functions.
- Hint: If-statement.

Overloading Constructors

- Constructors may sometimes be overloaded to initialize objects with a variety of initial values.
- Such constructors have the same name, but different signature (types of parameters, number of parameters or order of parameters)

```
class circle {  
    private:  
        double radius, area, circum;  
        const double pi;  
    public:  
        circle() : radius(0), pi(3.1416)  
        {  
            }  
  
        circle(double r) : pi(3.1416)  
        {  
            radius = r; }  
}
```



Overloaded Constructors: Example

```
class Distance                                //English Distance class
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)         //constructor (no args)
    { }

    Distance(int ft, float in) : feet(ft), inches(in) //constructor (two args)
    { }

}

int main()
{
    Distance dist1, dist3;                    //define two lengths
    Distance dist2(11, 6.25);                 //define and initialize dist2
}
```



The Copy Constructor

- Let's mention another way to initialize an object: you can initialize it with another object of the same type.
- Surprisingly, you don't need to create a special constructor for this; one is already built into all classes.
- It's called the **default copy constructor**.

```
int main()
{
    Distance dist1(11, 6.25);           //two-arg constructor
    Distance dist2(dist1);              //one-arg constructor
    Distance dist3 = dist1;             //also one-arg constructor
```



The Copy Constructor

- This causes the default copy constructor for the Distance class to perform a member-by-member copy of `dist1` into `dist2`.
- A different format has exactly the same effect, causing `dist1` to be copied member-by-member into `dist3`

```
Distance dist3 = dist1;
```



Copy Constructor: Shallow Vs Deep Copy

- In C++, a copy constructor typically makes a **shallow copy** by default.
- This means that it copies the values of member variables directly.
- If a class contains pointers or dynamically allocated memory, the default copy constructor will copy the pointer values.
- This leads to both the original and the copied object to point to the same memory.
- This can cause issues like double deletion or unintended side effects.



Copy Constructor: Shallow vs Deep Co

- For a **deep copy**, (where the copy constructor allocates new memory and copies the contents of the original object's pointed-to data) we need to define a copy constructor.
- Observe the syntax.

```
class MyClass {  
public:  
    int* data;  
  
    // Default constructor  
    MyClass(int value) {  
        data = new int(value);  
    }  
  
    // Custom copy constructor for deep copy  
    MyClass(const MyClass& other) {  
        data = new int(*other.data);  
        // Allocate new memory and copy the value  
    }  
  
    // Destructor  
    ~MyClass() {  
        delete data; // Clean up the allocated memory  
    }  
};
```



Member Functions Defined Outside Class

- In C++ you can define member functions outside the class.
- To do so, you should declare a function prototype (declaration) within the class.

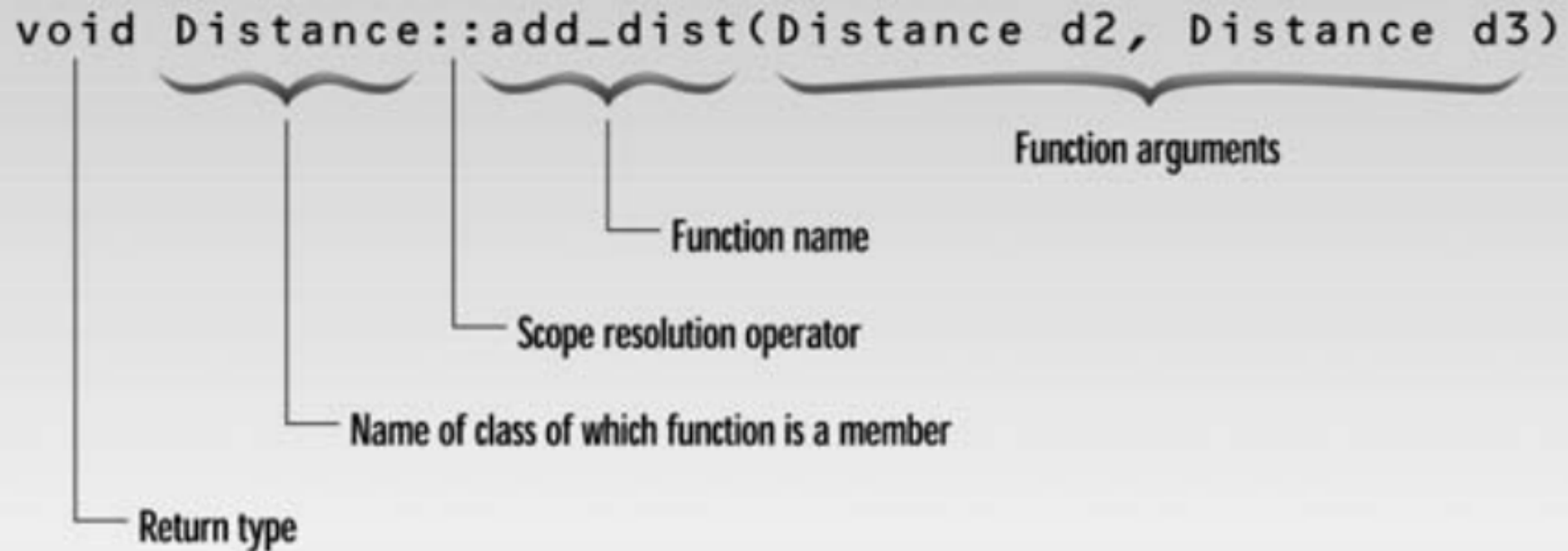
```
void add_dist( Distance, Distance );    //declaration
```

- The function can then defined outside the class as follows:

```
void Distance::add_dist(Distance d2, Distance d3)
```

- Observe the syntax.

Member Functions Defined Outside Class



The diagram shows the C++ member function definition `void Distance::add_dist(Distance d2, Distance d3)` with the following components labeled:

- Return type:** `void`
- Name of class of which function is a member:** `Distance`
- Scope resolution operator:** `::`
- Function name:** `add_dist`
- Function arguments:** `(Distance d2, Distance d3)`



Objects as Function Arguments

```
void Distance::add_dist(Distance d2, Distance d3)
{
    inches = d2.inches + d3.inches; //add the inches
    feet = 0;                       //(for possible carry)
    if(inches >= 12.0)               //if total exceeds 12.0,
    {                                //then decrease inches
        inches -= 12.0;             //by 12.0 and
        feet++;                     //increase feet
    }                               //by 1
    feet += d2.feet + d3.feet;      //add the feet
}
```



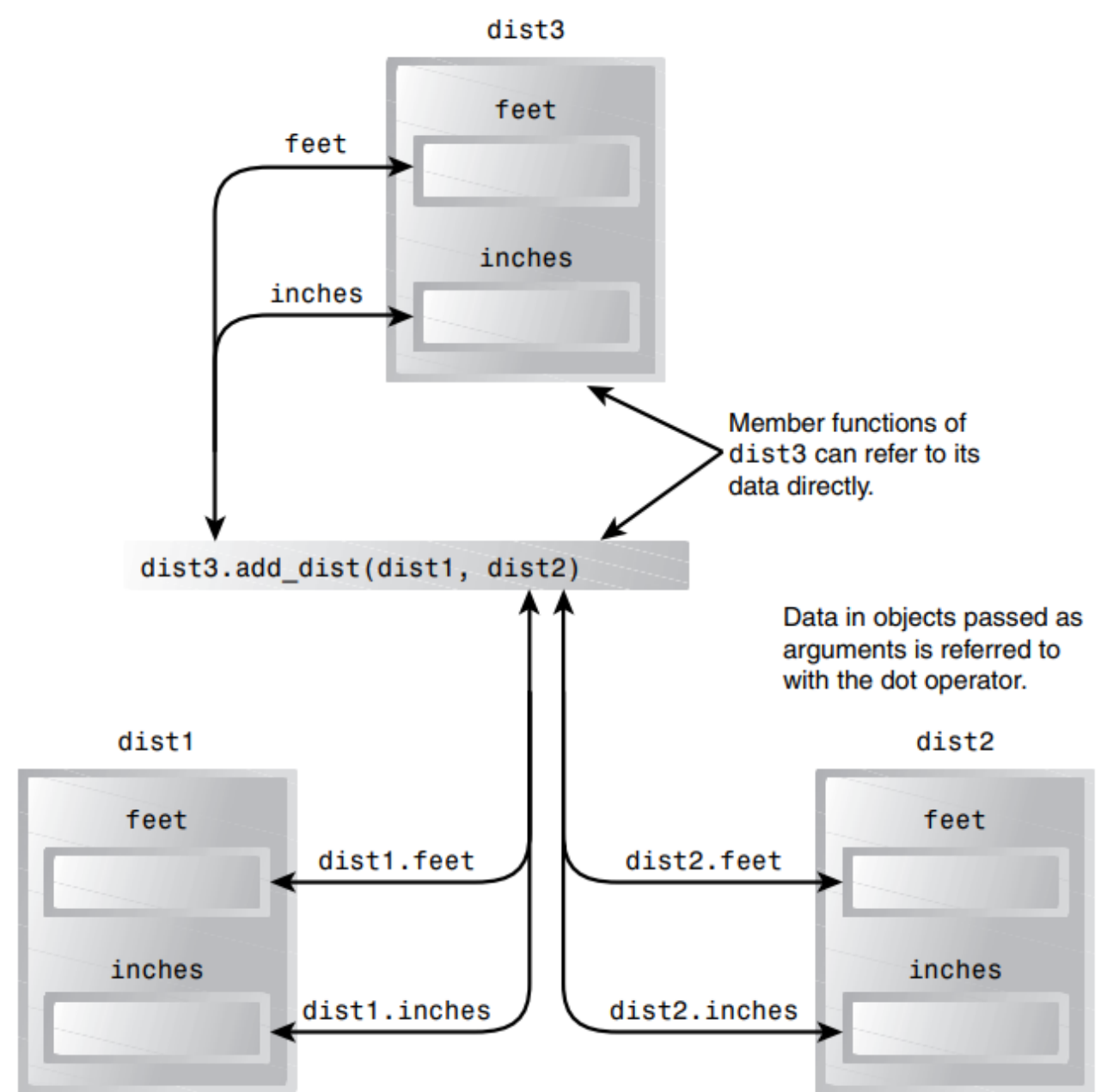
Objects as Function Arguments

Now we want to add `dist1` and `dist2` to obtain `dist3`. The function call in `main()`

```
dist3.add_dist(dist1, dist2);
```

- The two distances to be added, `dist1` and `dist2`, are supplied as arguments to `add_dist()`.
- The syntax for arguments that are objects is the same as that for arguments that are simple data types such as `int`:
- Since `add_dist()` is a member function of the `Distance` class, it can access the private data in any object of class `Distance` supplied to it as an argument, using names like `dist1.inches` and `dist2.feet`

Objects as Function Parameters





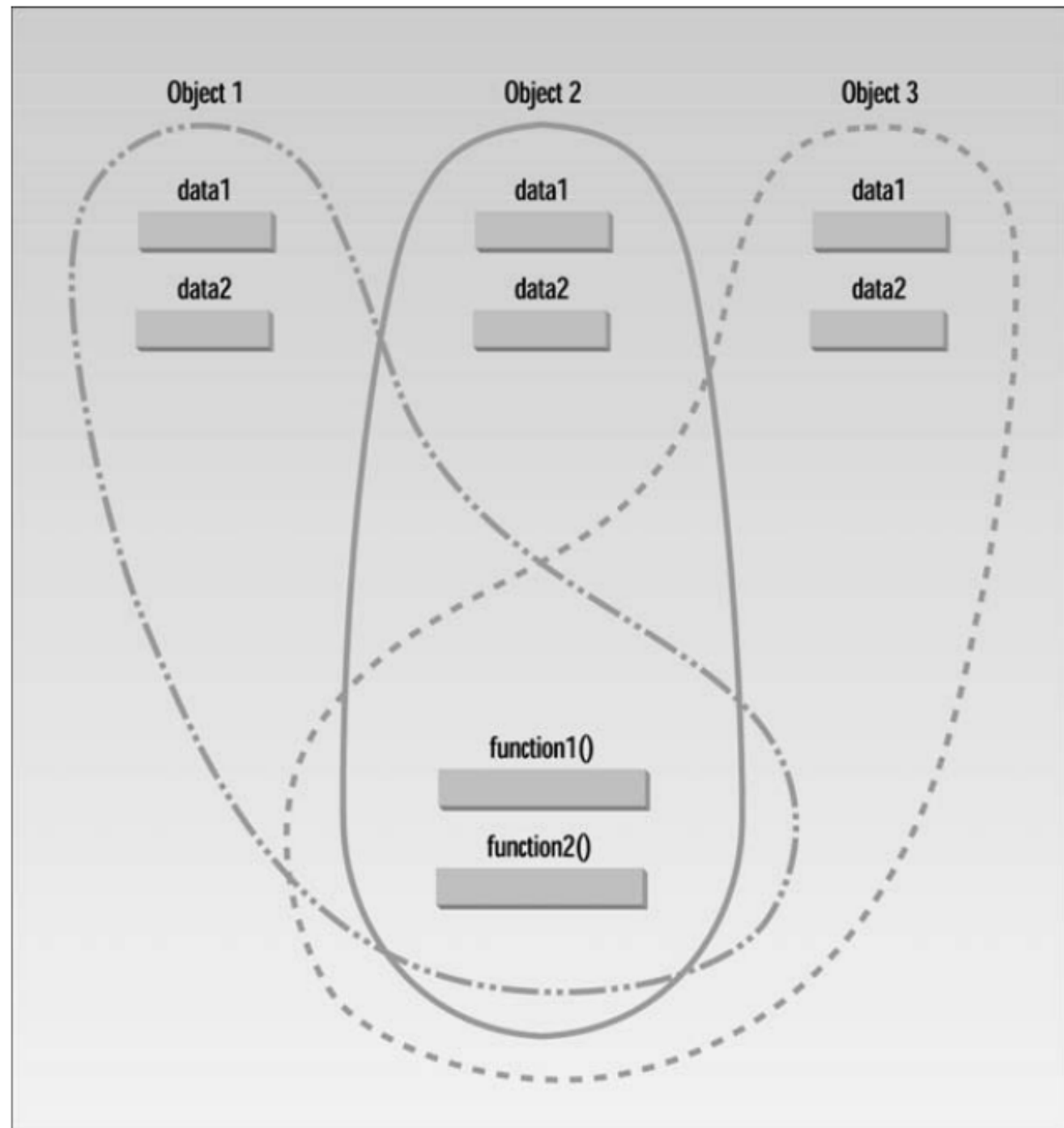
Returning Objects from Functions

```
//add this distance to d2, return the sum
Distance Distance::add_dist(Distance d2)
{
    Distance temp;                //temporary variable
    temp.inches = inches + d2.inches; //add the inches
    if(temp.inches >= 12.0)         //if total exceeds 12.0,
    {                               //then decrease inches
        temp.inches -= 12.0;       //by 12.0 and
        temp.feet = 1;            //increase feet
    }                               //by 1
    temp.feet += feet + d2.feet;   //add the feet
    return temp;
}
```

```
dist3 = dist1.add_dist(dist2);
```

Classes and Object Memory

- In terms of internal memory layout,
- All the objects in a given class use the same member functions.
- The member functions are created and placed in memory only once—when they are defined in the class definition.





Static Class Data

- If a data item in a class is declared as static, only one such item is created for the entire class, no matter how many objects there are.
- A static data item is useful when all objects of the same class must share a common item of information.
- A member variable defined as static has characteristics similar to a normal static variable:
- It is visible only within the class, but its lifetime is the entire program. It continues to exist even if there are no objects of the class.



Destructors

- In order to deallocate memory allocated to an object, a special member function called a *destructor* must be invoked.
- A destructor has the same name as the constructor (which is the same as the class name) but is preceded by a tilde:

```
class Foo
{
private:
    int data;
public:
    Foo() : data(0)           //constructor (same name as class)
    { }
    ~Foo()                   //destructor (same name with tilde)
    { }
};
```



Destructors – Some Rules

- It is not possible to define more than one destructor.
- The destructor is only one way to destroy the object created by the constructor. Hence, destructor cannot be overloaded.
- It cannot be declared static or const.
- Destructor neither requires any argument nor returns any value.
- It is automatically called when an object goes out of scope.
- Destructor release memory space occupied by the objects created by the constructor.
- In destructor, objects are destroyed in the reverse of an object creation.



Summary

- This session was a first look at Object Oriented Programming.
- A class can be considered as an extension of a structure – contains both data and functions on that data.
- Access Modifiers such as public and private control access to both data and functions.